

# Programmeren: algoritmen en structuren

De bouwstenen van een programma, de vier datastructuren, de algoritmische technieken, en correctheid en eindigheid.

---

## 1 Van probleem naar programma

**Een algoritme is een stappenplan dat na eindig veel stappen tot een oplossing komt.** Het bestaat los van de taal; pas daarna schrijf je het in **Python**, in een online omgeving, zodat je niets op je eigen computer hoeft te installeren.

Je werkt in **vier stappen: het probleem analyseren, een algoritme ontwerpen, het programmeren, en testen en debuggen**. Meteen beginnen typen zonder het probleem te analyseren kost je achteraf meer tijd dan het wint. **Debuggen is fouten opsporen en herstellen**, en je **test ook met randgevallen, omdat fouten zich daar verstoppen**: een lege lijst, één element, een nul, een negatief getal.

**Geef variabelen een zinvolle naam, zodat je code leesbaar blijft** voor jezelf en voor anderen. De computer kan het niets schelen, maar een lezer wel, en dat telt mee in de beoordeling. **Commentaar wordt niet mee uitgevoerd**; ze staat er alleen voor wie de code leest.

---

## 2 De bouwstenen

| Bouwsteen | Wat ze doet                                           | In Python                                          |
|-----------|-------------------------------------------------------|----------------------------------------------------|
| variabele | een naam waarachter een waarde zit die kan veranderen | je overschrijft de waarde gewoon                   |
| constante | een waarde die tijdens het programma niet verandert   | het aantal seconden in een uur, met een naam erbij |
| conditie  | een stuk code alleen laten lopen als iets waar is     | de constructie met als en anders                   |
| iteratie  | een herhaling van hetzelfde stuk code                 | een lus, over een lijst of zolang iets geldt       |
| functie   | een stuk code een naam geven en hergebruiken          | één keer schrijven, zo vaak oproepen als je wil    |

---

### 3 De vier datastructuren

**Een string is een rij tekens:** letters, cijfers en leestekens na elkaar, tussen aanhalingstekens.

| Datastructuur                                 | Waarvoor                                   | Kenmerk                                   |
|-----------------------------------------------|--------------------------------------------|-------------------------------------------|
| list                                          | een geordende rij die je nog wil aanpassen | geordend én aanpasbaar                    |
| tuple                                         | een geordende rij die vastligt             | je kan hem achteraf niet meer wijzigen    |
| set                                           | enkel de verschillende waarden overhouden  | elk element komt er maar één keer in voor |
| dictionary, in het Nederlands een woordenboek | paren van een sleutel en een waarde        | je zoekt iets op met de sleutel           |

**Een list en een tuple zijn allebei geordend; alleen de tuple ligt vast. Een set kan hetzelfde element niet twee keer bevatten:** steek je de getallen één, twee, twee en drie erin, dan blijven er **drie** elementen over. Dubbels eruit halen is precies waar een set voor dient.

---

### 4 Algoritmische technieken

**Recurisie is een functie die zichzelf oproept**, waarbij het probleem telkens een beetje kleiner wordt tot het vanzelf oplosbaar is. **Elke recursieve functie heeft een geval nodig waarin ze zichzelf niet meer oproept**, een stopgeval. **Zonder dat blijft ze zichzelf oproepen tot het programma vastloopt**, en geeft Python een foutmelding omdat de oproepen te diep gaan.

**Verdeel-en-heers** is de techniek waarbij je **een probleem opsplijt in kleinere deelproblemen en de deeloplossingen daarna samenvoegt**. Sorteren doe je zo: splits de lijst in twee, sorteer elke helft en voeg ze samen. Moet je in een **gesorteerde lijst van duizend getallen** nagaan of een getal erin staat, dan is de slimste aanpak **telkens de helft wegnemen**: tien stappen volstaan dan.

**Bij dynamisch programmeren bewaar je tussenresultaten zodat je ze niet twee keer hoeft te berekenen.** Bij de rij van Fibonacci scheelt dat het verschil tussen een seconde en een eeuwigheid. Het **kost meer geheugen maar wint tijd**.

---

## 5 Correctheid, eindigheid en vergelijken

**Een algoritme is correct als het voor elke toegelaten invoer het juiste antwoord geeft**, niet alleen voor de gevallen die je toevallig uitprobeerde. **Het is eindig als het na een eindig aantal stappen stopt. Je beargumenteert de eindigheid omdat een programma dat blijft lopen niets oplost**, ook al klopt elke stap: bij een lus toon je dat de voorwaarde ooit vals wordt, bij recursie dat je het stopgeval altijd bereikt.

**Twee oplossingen voor hetzelfde probleem vergelijk je op snelheid, geheugengebruik en gedrag bij veel gegevens. Twee algoritmen die hetzelfde antwoord geven, zijn daarom nog niet even snel**: het ene kan duizend keer trager zijn. Een oplossing die bij tien getallen vlot werkt, kan bij een miljoen getallen onbruikbaar worden.

---

## 6 Bestanden en bibliotheken

Naast een gewoon tekstbestand lees je ook **CSV**-bestanden in en schrijf je ze weg: bestanden waarin de waarden door komma's of puntkomma's gescheiden staan, zoals een tabel.

**Je mag matplotlib, numpy en random gebruiken**, en geen andere, tenzij de opdracht er uitdrukkelijk een vermeldt en toelicht. **Matplotlib dient om grafieken te tekenen** van je gegevens, **numpy om vlot met grote rijen getallen te rekenen** (veel sneller dan met een lus over een gewone list), en **random om toevalsgetallen te laten genereren**, handig voor een simulatie.

**Een numerieke methode is een manier om een antwoord te benaderen met rekenstappen**. Je krijgt geen exacte formule maar een benadering, die je zo nauwkeurig maakt als je wil. Zo **benader je een bepaalde integraal door de oppervlakte van heel veel smalle stroken op te tellen**: precies de Riemannsom, maar dan door de computer uitgevoerd.

---